# Cross-owner reuse of agent skills on GitHub: predictability and construction biases of GitSkills*

**Pedro Odorize**[1]

Institute of Informatics – Federal University of Rio Grande do Sul (UFRGS)
Porto Alegre, RS – Brazil

pedro.odorize@inf.ufrgs.br

**Abstract.**
Agent skills are copied between repositories without a central registry, and the GitSkills dataset documents this diffusion at scale. We investigate whether observable characteristics of a skill and of its repository are associated with reuse across distinct GitHub owners, and how much of this predictability is an artifact of how the dataset was built. The rule that selects the representative copy of each content makes its location class depend on the number of copies: the probability that the representative sits in `.claude/skills/` rises from 14% (2 repositories) to 92% (more than 50), while the actual share of copies in that folder stays between 5% and 13%. In addition, about a quarter of the reused skills (24.7% in the full population) exist under a single owner, and a few accounts concentrate the sample. After excluding the identified construction-dependent shortcuts, with owner-grouped validation, nested tuning and a new owner-disjoint holdout (25,011 skills from 10,484 owners), a Random Forest reaches an AUC of 0.73 (95% CI 0.72–0.75) for cross-owner reuse, and a HistGradientBoosting model reduces the RMSLE of the number of owners by 6% relative to the mean baseline (0.625 versus 0.664). The population is skill contents observed in at least two repositories: the study separates cross-owner replication from within-owner duplication and does not predict initial reuse among all skills. The estimates describe an owner-balanced sample (a cap per owner in validation, no concentrated owners in the holdout), not the whole ecosystem. The signal is modest, associative and cross-sectional (attributes and target come from the same snapshot, so this is not a prospective prediction of future reuse): the RMSLE gain is stable across protocol variations, whereas the AUC is sensitive to how concentrated owners are treated; skill and repository attributes contribute complementarily (an exploratory finding). The representative rule still decides which copy supplies the repository and path attributes, so residual selection effects cannot be ruled out.

**Keywords:** agent skills; software reuse; mining software repositories; construction bias; grouped validation.

*Data and code: `https://github.com/podorize/skillreuse`.

# 1 Introduction

## 1.1 Context and motivation

Agent skills are portable directories that combine structured metadata, natural-language instructions, and optional scripts and reference materials. At their core, a `SKILL.md` file specifies how an LLM-based agent should perform a task, including the constraints it must follow and the resources it may use [Agent Skills ]. Although software reuse has been extensively studied [Haefliger et al. 2008, Lopes et al. 2017], agent skills introduce distinct challenges because their distribution does not require a centralized registry or package management system. Instead, skills can be replicated by copying their directories across repositories. However, the presence of identical skill content in multiple repositories does not necessarily indicate independent adoption. Such replication may reflect characteristics of the skill itself, such as clarity and portability, or external factors, including its inclusion in templates or catalogs, automated installation by development tools, and duplication across projects maintained by the same account. Consequently, observed replication patterns reflect a combination of artifact characteristics and distribution contexts, making it difficult to distinguish potentially meaningful reuse from other forms of duplication.

The GitSkills dataset provides an opportunity to investigate these patterns at scale by documenting skill occurrences, distinct contents, and associated repository metadata [Destefanis et al. 2027]. Previous research has examined defects in skill specifications [Zhang et al. 2026], the effectiveness of skills in software engineering tasks [Han et al. 2026], and the evolution of prompts in software repositories [Tafreshipour et al. 2025]. Building on this literature, the present study investigates whether observable characteristics of agent skills and their repositories are associated with replication across distinct GitHub owners. It also examines a related methodological concern: whether the procedures used to construct the dataset introduce systematic biases that influence the apparent predictability of such replication.

## 1.2 Research problem

This study investigates **cross-owner replication**, defined as the presence of identical `SKILL.md` content, identified by its `content_hash`, in repositories belonging to at least two distinct GitHub owners. Each distinct content constitutes one observation, and an owner is defined as the account identified by the prefix of a repository's full name. The primary prediction target, `cross_owner`, indicates whether a content appears under two or more owners, while the secondary target, `owner_count`, measures the number of distinct owners associated with that content. Although counting owners provides a more meaningful approximation of diffusion than counting repositories, it does not establish independent adoption: multiple accounts may belong to the same individual or organization, and identical content may be propagated through templates, catalogs, or automated installation tools.

The analysis is restricted to contents observed in at least two repositories (`repository_count >= 2`). Within this population, the objective is to distinguish replication across owners from duplication within a single owner's repositories, rather

than to predict whether an arbitrary new skill will ever be reused. This restriction also explains the relatively high prevalence of cross-owner replication, which accounts for approximately 80% of the analyzed contents. Moreover, the study adopts a cross-sectional predictive design: both the attributes and the targets are derived from the same July 2026 repository snapshot. Consequently, repository characteristics such as stars, forks, and last-push dates may have been observed after replication occurred. The models therefore assess associations with observed replication rather than predicting future adoption. Accordingly, the findings are interpreted as predictive associations, without claims of causality or skill quality [Shmueli 2010].

Two methodological challenges further complicate the analysis. **The first concerns the measurement of reuse.** The number of repositories containing identical content (`repository_count`) may overstate diffusion because it includes copies maintained by the same owner. In the full population of contents appearing in at least two repositories, 24.7% occur exclusively under a single owner, a proportion that reaches 28.0% in the development set. Furthermore, one account accounts for approximately 12% of the development observations. These patterns indicate that repository counts conflate cross-owner replication with within-owner duplication, motivating the use of owner-based targets.

**The second challenge concerns the construction of predictive attributes.** GitSkills retains detailed metadata for only one representative occurrence of each distinct content. This representative is selected by prioritizing certain directory locations, particularly `.claude/skills/`, followed by a deterministic alphabetical ordering of repositories and paths. As the number of occurrences increases, so does the likelihood that at least one copy resides in a preferred location. Consequently, the representative's location class, along with the availability of commit-history information associated with that location, can encode information about the number of copies through the selection procedure itself. Such attributes may therefore produce misleading predictive signals unrelated to the characteristics that explain replication. To mitigate this problem, the main models exclude the identified construction-dependent variables, while their effects are examined separately. Nevertheless, the remaining repository and path attributes still describe the selected representative, so the exclusion does not eliminate every potential effect of the selection procedure.

Together, these challenges motivate a predictive framework that distinguishes cross-owner replication from within-owner duplication while accounting for identifiable dataset-construction biases. The evaluation combines owner-grouped validation, controls for owner concentration, and a separate owner-disjoint holdout. Because the research hypotheses and evaluation protocol were finalized after an initial exploratory phase, only the single evaluation of the frozen models on the new holdout is treated as confirmatory; results obtained during development remain exploratory.

### 1.3 Objectives and research questions

- **RQ1 — Construction biases:** to what extent do the representative-selection rule, self-copying and owner concentration confound the target and the attributes of GitSkills?

- **RQ2 — Predictability:** after excluding the identified construction-dependent shortcuts, can cross-owner reuse and the number of owners be predicted better than baselines, and is the estimate confirmed on an owner-disjoint holdout?
- **RQ3 — Origin of the signal:** what is the contribution of skill, repository and catalog attributes, which variables are associated with the predictions, and in which direction?

### 1.4 Contributions

(i) An **audit of the GitSkills representative**: we recover its rule (97.2% agreement) and show that location and history depend on the number of copies. (ii) A **protocol** for cross-owner reuse with an owner-based target, a per-owner cap, grouped validation, explicit hypotheses and confirmation on a new holdout. (iii) Evidence of a **modest predictive signal**, with a stable RMSLE gain across protocol variations, confirmed on unseen data (AUC 0.73) and decomposed into blocks (an exploratory decomposition). (iv) A **reproducible package** with the data revision, seeds, a five-minute reproduction path and a lock that prevents reusing the holdout.

## 2 Related work

### 2.1 Reuse, cloning and natural-language artifacts

Software reuse is studied as component adoption and as the dissemination of solutions across open ecosystems [Mockus et al. 2002, Haefliger et al. 2008], and large-scale cloning on GitHub shows that duplicated content is common [Lopes et al. 2017]; reuse may occur through dependencies, copying, templates and loosely standardized processes [Heinemann et al. 2011]. These works address mainly executable code. In agent skills, behavior is specified in natural language, and copying does not reveal whether the content was executed, adapted or adopted independently. We identify identical contents by hash, as the cloning literature does, but count distinct **owners** rather than repositories, to separate diffusion among developers from duplication by one account. The measure excludes modified copies; *reuse* is an operational term, implying no quality or adoption.

### 2.2 Agent skills and prompt artifacts

Research on LLMs with tool use studies selection, planning and execution of calls [Schick et al. 2023, Shen et al. 2023, Yao et al. 2023, Qin et al. 2023]. An agent skill is instead a persistent artifact, hosted in repositories, that describes procedures and conventions [Agent Skills ]. GitSkills provides the corpus [Destefanis et al. 2027]; other studies analyze specification and packaging defects [Zhang et al. 2026], the effect of skills on real engineering tasks [Han et al. 2026] and the evolution of prompts [Tafreshipour et al. 2025]. None asks whether observable characteristics of a skill and its context are associated with diffusion across owners, nor how much the construction of the corpus affects that question.

### 2.3 Validation, leakage and construction bias

When observations from the same project or organization appear in training and evaluation, a model may memorize the group instead of learning relations that generalize [Kaufman et al. 2012]; the literature recommends grouped cross-validation [Saeb et al. 2017, Roberts et al. 2017], and nested validation separates model selection from error estimation. Models may also exploit incidental regularities [Geirhos et al. 2020, Lapuschkin et al. 2019]. In mined data there is a variant: attributes produced by the collection or deduplication procedure itself may depend on the target by construction, without any column being derived from it. The choice of the representative in GitSkills is such a case. We treat it as a bias to be audited, and adopt owner-grouped validation, a per-owner cap, hypotheses fixed before the holdout and confirmation on disjoint data.

### 2.4 Modeling, explainability and research gap

We model cross-owner reuse (binary) and `log1p(owner_count)` (RMSLE) with six model families, with and without nested hyperparameter search. Explainability describes how a model combines attributes but does not turn associations into causal effects; permutation importance and partial dependence depend on the model, and correlated variables share importance [Molnar 2020, Shmueli 2010, Breiman 2001]. Our contribution is empirical and methodological for this corpus: to audit the representative bias, to redefine the target in terms of owners, and to estimate, with a pre-specified protocol and a new holdout, how much signal remains.

## 3 Methodology

### 3.1 Research design

The study is empirical, quantitative and predictive, with an interpretive layer; it measures predictive information and does not estimate causal effects [Shmueli 2010]. The **development** phase (exploratory) comprises the dataset audits, the definition of targets and feature views, grouped validation and model selection. The **confirmation** phase evaluates, once, the frozen models on a new holdout, with success criteria fixed beforehand. The full protocol, including decisions made after seeing results, is in `paper/protocol.md` (Portuguese).

### 3.2 Data and unit of analysis

The source is GitSkills [Destefanis et al. 2027], revision `ebab17454a7c236f8f26b183567f1a126f42e3f8`: 3,797,117 `SKILL.md` files from 282,200 repositories (July 2026), grouped into 1,877,981 distinct contents (`content_hash`). All occurrences are kept, but only one **representative** per content receives full text, repository metadata, folder listing and history. Development uses 200,000 contents chosen by `md5(content_hash || ':20261009')`, split by repository into 160,000 for development and 40,000 for test; we condition on `repository_count >= 2`, which leaves 25,918 contents from 7,128 owners in

development. In the full population, 306,088 contents satisfy this condition, 230,355 of them under two or more owners. The original test partition was used in exploratory analyses and supports no confirmatory claim. The reference date for age and recency is 2026-07-20.

### 3.3 Representative rule and audit

GitSkills documents that the representative prefers `.claude/skills/`, with a deterministic tie-break [Destefanis et al. 2027]. We recovered the rule from all raw occurrences: sort by location class (`canonical`, which is essentially `.claude/skills/` in any letter case, then `skills-dir`, then `other`) and, on ties, by the smallest `(repo_full_name, path)`. It reproduces the representative in 97.2% of the 306,088 contents with at least two repositories, and the representative's class equals the best available class in 100% of them. The audit compares, by number of repositories, the fraction of contents whose representative is `canonical` with the fraction of copies that actually are, to decide whether `location_class` and `history_fetched` (history was collected only for standard locations and a sample) can be legitimate predictors.

### 3.4 Targets

The main target is `cross_owner = owner_count >= 2`, where `owner_count` is the number of distinct owners (prefix of `repo_full_name`) across all occurrences; the secondary target is `log1p(owner_count)`, and `log1p(repository_count)` is used for sensitivity. Counting owners removes self-copying among repositories of one account. Counts are lower bounds, since GitHub code search indexes only the default branch, files up to 384 KB and recent repositories [Destefanis et al. 2027].

### 3.5 Features

**Table 1: Feature blocks. The main models use the first four (52 columns, the *clean* view); shortcuts enter only as a finding.**

| Block | n | Content |
|---|---|---|
| `skill` | 34 | structure of `SKILL.md` (size, headings, lists, code, links, front matter, scripts, folder) and file name |
| `repo` | 8 | `log1p` of stars and forks, fork flag, age, days since last push, active days, language, license |

| Block | n | Content |
|---|---|---|
| `catalog` | 2 | `log1p` of the number of skills in the repository; indicator of at least 10 |
| `path` | 8 | depth, root, `templates`, `plugins`, `vendor`, `docs`, `packages/src`, other-tool directory |
| *shortcuts* | 4 | `location_class`, `history_fetched`, `commit_count`, `.claude` in the path |

The shortcuts depend on the representative rule and are kept out of the main models. The attributes describe the representative, not all copies; aggregating over all copies would use the same occurrences that define the target and was avoided. Imputation (median with missing indicator), scaling and one-hot encoding (`min_frequency=10`) are fitted within each fold. Decisions on transformations (`log1p` for linear models only), redundant and constant columns and class balance follow from the feature EDA (Section 4.1). We do not resample the 80/20 imbalance of `cross_owner`: the main metrics are threshold-free, and PR AUC is read against prevalence.

### 3.6 Validation and per-owner cap

Validation is grouped by the **owner of the representative repository**: 3 repetitions of 3 folds (`StratifiedGroupKFold`, seeds 2027–2029, strata by `owner_count` bin), with the same partitions for all models. Because a few accounts concentrate the sample, we cap the contribution at **50 contents per owner** (in `content_hash` order), leaving 17,733 contents. The value 50 was chosen *after* observing the AUC's sensitivity to how concentrated owners are treated (Section 4.4); it is a data-dependent decision. Per fold we also measure the share of validation rows whose front-matter `name` appears in training and the metrics on the subset without overlap (a joint owner–`name` grouping forms a component with 64% of the rows and was discarded).

**Target population.** The reported AUC does not characterize the whole ecosystem. It describes performance under an owner-balanced sampling protocol: contents with at least two repositories, each owner contributing at most 50 contents to validation (an unweighted cap, not a reweighting), and, for the holdout, owners with at most 37 contents each. Skills of heavily concentrated owners are underrepresented by design, and no weighting scheme restores their share of the ecosystem.

### 3.7 Models, baselines and tuning

The main model is the Random Forest (100 trees for regression, 300 for classification; `max_depth=12`, `min_samples_leaf=3`, `max_features=sqrt`), with Ridge ($\alpha = 10$) and logistic regression ($C = 1$) as linear references and the prevalence and the mean and median of `log1p` as baselines. Tuning is nested: six families (linear, KNN, tree, Random Forest, Gradient Boosting, HistGradientBoosting) in two variants (default and `RandomizedSearchCV`) on the clean view, with an outer validation of 2 repetitions of 3 owner-grouped folds and an inner search of 3 owner-grouped folds and 6 candidates (planned: 3 repetitions and 12 candidates; reduced for computational cost). The combination with the best outer mean is frozen.

### 3.8 Metrics, inference and interpretation

The main metrics are ROC AUC (classification) and RMSLE (regression); the secondary ones are PR AUC, Brier score, log-loss, MAE and Spearman. Comparisons are paired by (repetition, fold), with 95% CIs and a t-test with the **Nadeau–Bengio correction** for repeated samples. Five hypotheses, formulated after the exploratory phase and before the confirmation, form a family corrected by Holm ($\alpha = 0.05$); all other comparisons are exploratory: **H1** the clean attributes predict cross-owner reuse (AUC against 0.5); **H2** they reduce the RMSLE of `owner_count` (clean RF against the mean of `log1p`); **H3a/H3b** skill adds to repository+catalog and vice versa (AUC); **H4** the shortcuts add to the clean attributes (RMSLE).

Interpretation uses permutation importance of the clean RF on owner-grouped folds (per column and per block) and out-of-fold partial dependence of the frozen models (trained on each training fold, curves computed on the validation rows). Both are associative. For the binary task we also report the minority class (`cross_owner = 0`) and the performance at a threshold that maximizes the balanced accuracy on development out-of-fold predictions; the threshold is then applied, unchanged, to the saved holdout predictions, as a descriptive reading and not a new evaluation.

### 3.9 New holdout

The holdout gathers contents outside the 200,000 sampled, with `repository_count >= 2`, whose owners do not appear among the reused skills of the sample (development and original test), with the same cap of 50: 25,011 contents from 10,484 owners, sharing no owners or contents with the sample (at most 37 contents per owner, so the cap of 50 does not actually bind in the holdout; its balance comes from the exclusion of concentrated owners). Only the prevalence of `cross_owner` (80.8%) was inspected, as an integrity check. The chosen models are trained on the whole development set and evaluated **once**, with CIs by owner bootstrap (2,000 resamples). Success requires the lower AUC bound above 0.5 and the upper bound of the RMSLE difference to the baseline below zero; a lock prevents re-evaluation without explicit declaration.

## 4 Results

**Roadmap.** Sections 4.1–4.2 answer RQ1 (dataset construction introduces identifiable biases); 4.3 and 4.7 answer RQ2 (a modest predictive signal remains under the clean protocol, first on development, then on the holdout); 4.5 answers RQ3 (skill and repository attributes carry complementary information, and which variables are associated with the predictions); 4.4 and 4.6 are robustness and model-selection checks, and the conclusions are conditional on the sampling and validation choices described in Section 3.

### 4.1 The target, the features and owner concentration

In development (25,918 reused contents from 7,128 owners), `owner_count` mirrors `repository_count`: mean 4.50 against 5.08, median 2 in both, P95 15 against 16, maximum 379 against 390, variance-to-mean ratio 23.1 against 22.1. The difference is at the base: **28.0%** of the contents have `owner_count = 1` (24.7% in the full population: 75,733 of 306,088), so self-copying accounts for 11% of the total mass of `repository_count` in development. A few owners concentrate the sample: the five largest account for 11.9%, 4.7%, 3.5%, 2.2% and 2.2% of the contents (24.5% together; identifiers omitted), and the largest and the fourth have on average 2.0 repositories but only 1.0 owner per content. In addition, 17.2% of the contents have no front-matter `name`, and 20.1% share it with another content, a sign of variants of the same skill.

**Features.** The clean view has 52 columns (49 numeric, 3 categorical) that expand to 112 after preprocessing. Missing values are rare (13 columns, at most 0.2%) and imputed by the training median with an indicator; one column (`repo_is_fork`) is constant and was kept as harmless. Twenty-five numeric columns have skewness above 2 (for example `sibling_count` 116, `sibling_bytes` 63, reduced to 1.6 and 0.5 by `log1p`); since trees are invariant to monotone transformations, we apply `log1p` only in a sensitivity analysis for linear models (Section 4.4). Eight pairs of columns have $|\rho| > 0.95$ (for example `body_chars` and `text_char_count`, 0.998); we kept them, since trees tolerate redundancy, and read the importance of redundant columns with caution. No dimensionality reduction was applied. Finally, `cross_owner` is positive for 80.4% of the development contents (80.8% in the holdout), and we do not resample.

### 4.2 Selection bias of the representative

The table compares, on the full population (1,877,981 contents), the fraction of contents whose representative is `canonical` with the fraction of copies that actually are.

**Table 2: Effect of the representative rule, by number of repositories (full population).**

| Repositories | contents | representative is `canonical` | copies in `canonical` |
|---|---|---|---|
| 1 | 1,571,893 | 13.3% | 12.6% |
| 2 | 172,624 | 13.8% | 8.5% |
| 3 | 49,535 | 19.5% | 9.1% |
| 4–5 | 36,312 | 24.1% | 10.9% |

| Repositories | contents | representative is `canonical` | copies in `canonical` |
|---|---|---|---|
| 6–10 | 24,142 | 30.4% | 10.1% |
| 11–50 | 19,589 | 57.7% | 7.7% |
| more than 50 | 3,886 | 91.7% | 5.1% |

The probability that the representative is `canonical` grows from 14% to 92% with the number of copies, while the actual share of copies in that folder is flat or decreasing (5–13%): a single copy in `.claude/skills/` is enough, by the recovered rule, for the representative to be `canonical`. The representative's location class, and the commit history that exists only for standard locations, therefore **depend on the number of copies**. A control with the target `repository_count` shows the size of the effect: with the Random Forest, the repository attributes gave an RMSLE of 0.575, but 0.604 without `history_fetched` and `commit_count` (difference +0.029, 95% CI [0.018; 0.040]), against 0.623 for the mean-of-`log1p` baseline; the location class alone (three categories) gave 0.584.

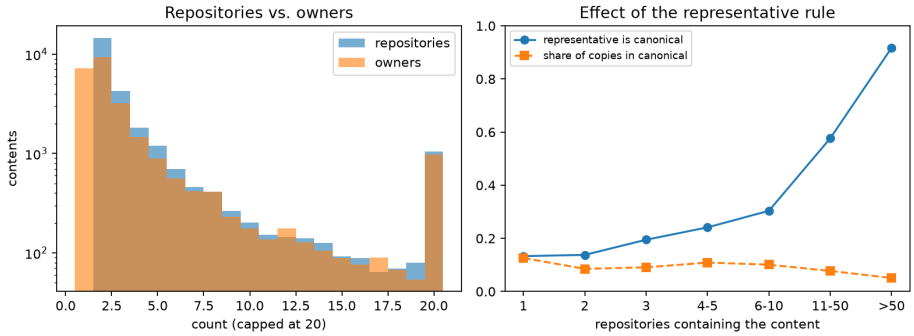



**Figure 1: Distributions of repositories and owners (left) and effect of the representative rule (right)**

### 4.3 Main protocol

With the shortcuts out, 17,733 contents (cap of 50 per owner) and owner-grouped validation, the results are:

**Table 3: Performance by view (mean ± standard deviation over 9 owner-grouped folds).**

| View | RF AUC | logistic AUC | RF RMSLE | Ridge RMSLE |
|---|---|---|---|---|
| skill | 0.660 ± 0.022 | 0.614 | 0.698 ± 0.007 | 0.713 |
| repository | 0.650 ± 0.019 | 0.655 | 0.707 ± 0.011 | 0.702 |
| repository + catalog | 0.666 ± 0.009 | 0.662 | 0.696 ± 0.010 | 0.699 |
| path + catalog | 0.579 ± 0.019 | 0.576 | 0.718 ± 0.018 | 0.714 |
| **clean (full)** | **0.716 ± 0.024** | 0.680 | **0.679 ± 0.011** | 0.694 |

| View | RF AUC | logistic AUC | RF RMSLE | Ridge RMSLE |
|---|---|---|---|---|
| clean + shortcuts | $0.717 \pm 0.022$ | 0.682 | $0.637 \pm 0.015$ | 0.664 |
| shortcuts only | $0.500 \pm 0.015$ | 0.510 | $0.688 \pm 0.010$ | 0.693 |
| *baseline* | 0.500 | 0.500 | 0.720 (mean), 0.780 (median) | |

**Table 4: Hypotheses formulated before the confirmation, evaluated on development (exploratory until Section 4.7).**

| | Comparison | Result | 95% CI of the difference | p (Holm) |
|---|---|---|---|---|
| H1 | clean RF AUC against 0.5 | 0.716 | +0.216 [0.173; 0.259] | $< 0.001$ |
| H2 | clean RF RMSLE against the mean | 0.679 vs 0.720 | $-0.041$ [$-0.049$; $-0.033$] | $< 0.001$ |
| H3a | clean against repository+catalog (AUC) | 0.716 vs 0.666 | +0.050 [0.017; 0.083] | 0.008 |
| H3b | clean against skill (AUC) | 0.716 vs 0.660 | +0.056 [0.040; 0.072] | $< 0.001$ |
| H4 | clean+shortcuts against clean (RMSLE) | 0.637 vs 0.679 | $-0.042$ [$-0.055$; $-0.029$] | $< 0.001$ |

The 95% intervals are unadjusted (paired differences with the Nadeau–Bengio variance correction); the p-values are Holm-adjusted over H1–H4. All five comparisons are rejected after Holm correction, and none of the conclusions depends on the adjustment alone, since every interval excludes zero. There is a real but modest signal (AUC 0.72, a 6% RMSLE reduction). Skill and repository contribute **complementarily**: each block alone sits at AUC 0.65–0.67 and the combination at 0.72 (H3a, H3b). The shortcuts add to the regression (H4) but alone do not predict cross-owner reuse (AUC 0.50): their regression gain comes from correlation with the number of copies, not from cross-owner reuse.

### 4.4 Robustness

**Table 5: Sensitivity to the treatment of concentrated owners (RF; gain relative to the mean-of-`log1p` baseline).**

| Treatment of concentrated owners | contents | clean RF AUC | RMSLE gain (clean) |
|---|---|---|---|
| no limit | 25,918 | $0.747 \pm 0.140$ | 0.039 |
| exclude owners with more than 1000 | 21,615 | $0.615 \pm 0.036$ | 0.036 |
| exclude owners with more than 500 | 19,567 | $0.708 \pm 0.037$ | 0.030 |
| exclude owners with more than 200 | 18,443 | $0.732 \pm 0.035$ | 0.037 |
| **50 per owner** | 17,733 | $0.716 \pm 0.024$ | 0.041 |

The RMSLE gain is stable (0.03–0.04), but the AUC oscillates from 0.615 to 0.747: a few owners with 500 to 1000 contents lower it by 0.1, and with no limit one fold concentrates one owner and the standard deviation reaches 0.14. The cap of 50 stabilizes the estimates but is a data-dependent choice.

Other checks. (i) Fixing the location class and training within each one (owners with more than 500 contents excluded), the clean RF still beats the baseline: `canonical` (5,439 contents) RMSLE $-0.141$, AUC 0.714; `skills-dir` (10,721) $-0.013$, 0.672; `other` (3,810) $-0.011$, 0.828. (ii) With `repository_count` as target, the gain is the same ($-0.031$ against $-0.030$). (iii) Only 18.8% of validation rows have a `name` seen in training, and the gain persists without them (RMSLE $-0.039$ against $-0.041$; AUC 0.726 against 0.716). (iv) Repository-grouped validation, without the cap, gave no larger estimates than owner-grouped validation (RMSLE 0.657 against 0.655; AUC 0.735 against 0.747). (v) Applying `log1p` to the 25 skewed columns improves the linear models only slightly (logistic AUC 0.680 $\rightarrow$ 0.691; Ridge RMSLE 0.694 $\rightarrow$ 0.690) and leaves the forests unchanged; the forest's advantage over the linear model remains indistinguishable from zero (AUC +0.023, 95% CI $[-0.012; 0.058]$).

### 4.5 Interpretation

For the clean RF (permutation on 6 owner-grouped folds, owners with more than 500 contents excluded), for cross-owner reuse, the repository block weighs more (AUC drop of 0.104 when permuted) than skill (0.084), catalog (0.016) and path ($\approx 0$); among columns, `log_stars` dominates (0.056), followed by `log_forks` (0.021) and `push_recency_days` (0.016). In regression, repository (0.019) and skill (0.018) weigh similarly, and the largest column is the license (0.013), followed by catalog size (0.011) and path depth (0.005). These values are the largest measured permutation-importance decreases under the evaluated model and data distribution, not the determinants of reuse. Uncertainties are large (6 folds), and stars and forks are correlated (as

are several skill-structure attributes), so permutation can split or shift importance among them unpredictably: the ranking between `log_stars` and `log_forks`, in particular, should not be read as separate effects.

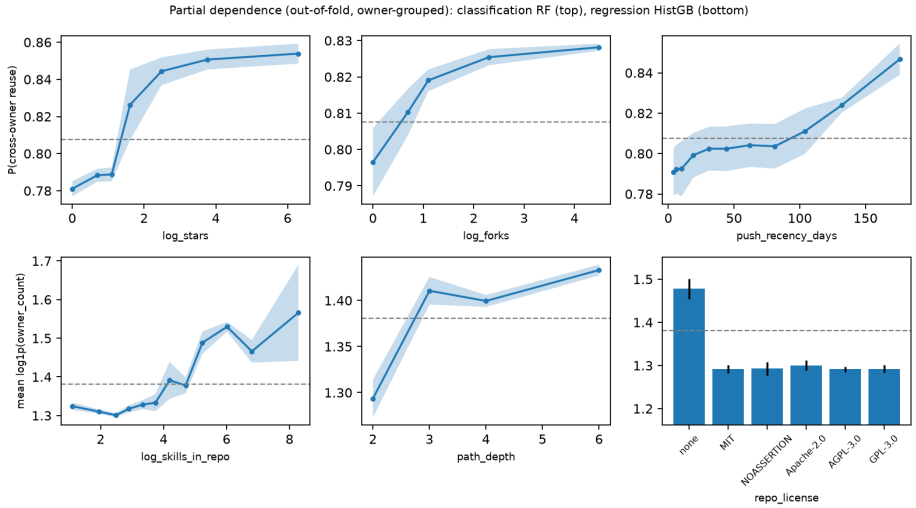



**Figure 2: Out-of-fold partial dependence of the frozen models: classification RF (top) and regression HistGB (bottom); bands are ±1 standard deviation across folds, dashed lines the mean observed prediction**

The partial dependence shows the **direction** of the model's response, and may evaluate combinations of attributes that are rare in the data (for example many forks with no stars). The probability of cross-owner reuse rises with stars (0.78 with none to 0.85 at about 540, mostly between 1 and 12 stars) and forks (0.80 to 0.83), and also with the **time since the last push**: repositories pushed more than about 100 days ago have higher probability (0.79 at 4 days, 0.85 at 176). In regression, the expected number of owners rises with catalog size (clearly only above roughly 60 skills) and with path depth (a jump from 2 to 3). The unexpected result is the license: what matters is its **absence**. Repositories without a license (52% of the development contents, median of zero stars) host skills with 6.6 owners on average, against 3.7 for MIT, and the model predicts 1.48 against 1.29–1.30 for `log1p(owner_count)` for any license; they also have more `canonical` skills (34% against 24%). This is an exploratory finding, not an explanation of reuse. We do not know the mechanism: it may reflect repositories created automatically or by installers, and the data do not separate this from an effect of the missing license itself.

### 4.6 Model selection and tuning

**Table 6: Nested validation (mean of 6 outer owner-grouped folds; 6 candidates per search).**

| Family | AUC default | AUC searched | RMSLE default | RMSLE searched |
|---|---|---|---|---|
| linear | 0.678 | 0.689 | 0.692 | 0.690 |
| KNN | 0.667 | 0.690 | 0.703 | 0.686 |
| tree | 0.614 | 0.634 | 0.807 | 0.699 |
| Random Forest | **0.716** | 0.701 | 0.679 | 0.675 |

| Family | AUC default | AUC searched | RMSLE default | RMSLE searched |
|---|---|---|---|---|
| Gradient Boosting | 0.691 | 0.704 | 0.690 | 0.671 |
| HistGradientBoosting | 0.693 | 0.682 | **0.670** | 0.688 |
| *baseline* | 0.500 | | 0.720 | |

By the pre-specified rule we froze the default RF (classification) and the default HistGradientBoosting (regression). **Under the evaluated configurations, the families are not clearly separated:** the RF against the runner-up (searched Gradient Boosting) differs by 0.012 AUC (95% CI [−0.003; 0.026]), and in regression the HistGB differs from the searched Gradient Boosting by −0.001 [−0.027; 0.024]. Only the tree (and, with less evidence, the default KNN) falls behind, and in classification even these differences do not survive Holm correction (p between 0.06 and 0.13). Intervals here are unadjusted 95% intervals; Holm adjusts only the p-values, within the family of comparisons of the best configuration against all others and the baseline. Non-significance reflects the limited resolution of 6 outer folds and does not establish equivalence. The search did not beat the default configuration (RF: 0.716 against 0.701).

### 4.7 Confirmation on the holdout

The frozen models were evaluated once on 25,011 contents from 10,484 owners disjoint from development.

**Table 7: Single evaluation on the holdout.**

| Task | Model | Result | Baseline | 95% CI (owner bootstrap) |
|---|---|---|---|---|
| `cross_owner` | Random Forest | AUC **0.731**; PR AUC 0.913; Brier 0.140; log-loss 0.444 | 0.5; 0.808; 0.155; 0.490 | AUC [0.716; 0.745]; PR AUC [0.905; 0.921] |
| `log1p(owner_count)` | HistGradientBoosting | RMSLE **0.625**; Spearman 0.330 | 0.664 (mean of `log1p`) | difference −0.038 [−0.044; −0.032] |

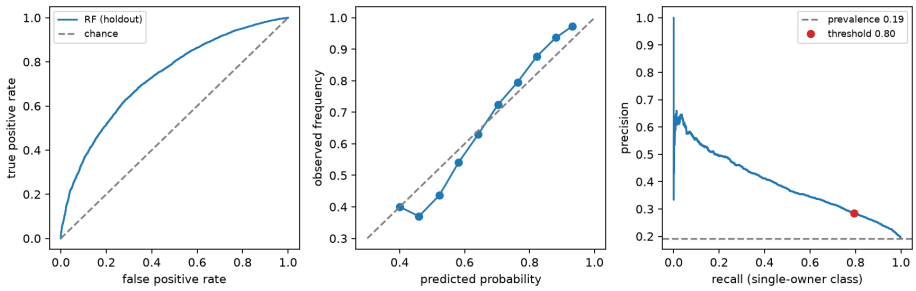



**Figure 3: ROC curve, calibration and precision–recall curve of the single-owner (minority) class for the RF on the holdout**

Both success criteria were met, and the holdout AUC (0.731) is consistent with the nested estimate on development (0.716). The holdout has a different composition (36% `canonical` skills against 21% in development, 9% `other` against 21%), since concentrated owners are left out by construction; H3 and H4 were not re-evaluated and remain exploratory.

**Minority class and threshold.** The PR AUC above refers to the majority class (80.8% prevalence), so it says little. For the minority class (content under a single owner, 19.2%), the PR AUC is 0.390 (95% CI [0.362; 0.418]), about twice its prevalence. The threshold that maximizes the balanced accuracy on development out-of-fold predictions is 0.80 (the default 0.5 gives only 0.506 there). Applied unchanged to the holdout, it gives a balanced accuracy of 0.659 (95% CI [0.647; 0.670]), against 0.663 on development, with a recall of 0.79 and a precision of 0.28 for the single-owner class (3,816 of 4,809 true single-owner contents recovered, but 9,607 cross-owner contents flagged as single-owner). The model therefore **orders** contents usefully, but a threshold produces many false alarms, and it should not be used for automated decisions.

## 5 Discussion

### 5.1 What the signal means

Observable attributes of a skill and of its repository carry information about cross-owner reuse, but not much: an AUC of 0.73 on the holdout and a 6% RMSLE reduction relative to the mean of `log1p`. The result holds across three layers of evidence (nested validation, development robustness and an owner-disjoint holdout), but it is a signal of **coarse ordering**: the Spearman of the regression model is 0.33, and at the threshold that maximizes balanced accuracy the model recovers 79% of single-owner contents with a precision of 28%.

The block decomposition answers RQ3: skill (file structure) and repository (visibility and activity) contribute complementarily (AUC 0.65–0.67 each, 0.72 together). Permutation importance and partial dependence associate cross-owner reuse mainly with **stars and forks** (stars give the largest permutation-importance decrease in the evaluated model, but the two are correlated and their individual shares are not separable) and with repositories whose **last push is old**, which is consistent with skills in more visible repositories, and in repositories that have been inactive for longer, being copied by other owners. The association admits more than one explanation (visibility that generates copies, accumulated exposure time, or repositories valued for containing skills that were already copied), and the data do not separate them. In the regression, what stands out is the **absence of a license** and, for large catalogs, the number of skills in the repository; the license result is an exploratory finding with unknown mechanism (Section 4.5), not a general explanation of reuse.

**Cross-sectional, not prospective.** Attributes and labels come from the same July 2026 snapshot. Stars, forks and last-push date were measured after some or all observed copying could have occurred, so the association with stars and forks may reflect copies that raised visibility as much as visibility that attracted copies, and an old last push may reflect accumulated exposure time. The results therefore show that attributes discriminate

between skills with different levels of observed replication; they do not show that these attributes would have predicted reuse before it happened. This is not target leakage in the conventional sense (no attribute is computed from the target), but it separates our problem from prospective prediction, which would require attributes dated before the copies appear.

### 5.2 Implications for using GitSkills

The most solid contribution is methodological and answers RQ1. (1) **`location_class` and `history_fetched` are not legitimate predictors of reuse**: by construction, the representative's class grows with the number of copies (14% $\rightarrow$ 92%) and history inherits the dependence; including them makes the "repository" view look much more informative than it is (Section 4.2). (2) **Counting repositories mixes diffusion with self-copying**; the number of owners is a more faithful target (a quarter of the reused contents have a single owner). (3) **Results depend on a few accounts**: the AUC varied from 0.615 to 0.747 with the treatment of owners with 500 to 1000 contents, so sensitivity to concentrated owners, or a cap per owner, should be standard practice. (4) **The representative describes an arbitrary copy**: among copies of the same class the choice is alphabetical, so repository attributes are those of one repository among several. Aggregates over all copies would be more informative but derive from the same occurrences that define the target; they should be treated as leakage in predictive studies, or computed only from information prior to reuse.

### 5.3 Models and data limits

Under the evaluated configurations, the six families are not statistically separated in either task (apart from the tree and, less clearly, KNN), and the hyperparameter search did not help (default RF 0.716 against 0.701 searched). The linear models trail the forests by 0.02–0.04 AUC even after `log1p`, but with intervals that include zero: the evidence is of a tie within the resolution of 6 outer folds, not of proven equivalence. Under the families and the tuning budget evaluated (6 outer folds, 6 candidates per search, reduced for computational cost), increasing model flexibility did not consistently improve performance. The limited resolution of the comparison means that small real differences may go undetected, and the data do not show that substantially better performance is impossible with more careful model selection, other architectures, more extensive optimization or alternative representations. A plausible reading is that the attributes describe one copy and the target is an exact count of hashes that ignores modified copies, so new information (relations among copies, time, richer representations) may be a more promising direction than more flexible models, but this remains a hypothesis.

### 5.4 Threats to validity and limitations

**Construct and temporal.** The study is cross-sectional (Section 5.1): it does not assess prospective prediction of future reuse. The number of owners approximates independent adoption only partly: it removes duplication by one account, but templates, catalogs and installation tools can still generate replication among owners. Forks are not a concern

in these data (an audit found 8 occurrences in 3 fork repositories out of 3,797,117, none changing a `cross_owner` label; `results/owner/fork_audit.json`). but accounts may represent the same person or organization, and GitHub search counts forks only if they have more stars than the parent. Counts are lower bounds, the exact hash misses variants, and reuse by copying does not indicate use, quality or usefulness.

**Internal.** (i) H1–H4 were fixed after exploratory results; only the holdout is confirmatory. (ii) The cap of 50 per owner is data-dependent. (iii) Tuning used 2 repetitions of 3 folds and 6 candidates, fewer than planned. (iv) With 9 non-independent scores, the Nadeau–Bengio test is approximate, and Holm p-values depend on the family size. (v) **Residual selection effects:** excluding the shortcuts does not remove the influence of the representative rule, which still decides which repository supplies stars, forks, license, last push and the catalog and path attributes, and the number of copies may influence that selection; a robustness check with a representative chosen independently of location would require attributes of alternative copies and was not feasible here. The catalog and path blocks may be partly affected by the preference for standard locations; that the path barely helps and that the signal persists within each location class reduces, but does not eliminate, this risk. (vi) The threshold analysis is descriptive: it uses the saved holdout predictions.

**External.** The population is contents with at least two repositories, so the results do not apply to whether a skill is reused at all. The reported AUC is performance under the owner-balanced protocol (Section 3.6), not an unrestricted ecosystem-wide estimate: the cap of 50 in development was chosen after seeing the AUC's sensitivity (0.615 to 0.747), and the holdout contains no concentrated owners (at most 37 contents each), so the result does not generalize to heavily concentrated owners, whose behavior may differ. Inverse-owner-frequency weighting, instead of a cap, is a natural comparison for future work. The corpus is a July 2026 snapshot of public repositories. The holdout excludes concentrated owners by construction and has more `canonical` skills (36% against 21%) than development. The original test partition was used in exploratory analyses and supports no confirmatory claim. Near-identical variants of the same skill may still lie on both sides of the split.

**Interpretation and ethics.** Permutation importance and partial dependence are associative; correlation among attributes (stars and forks, several structure counts) can distribute importance unpredictably, and partial dependence can evaluate rare combinations, so strongest-feature statements are specific to the evaluated model and data distribution. They depend on the model and vary across folds; nothing here establishes causality. The data are public and GitSkills anonymizes commit authors; we omit owner identifiers, and the analysis does not aim to evaluate individuals.

## 6 Conclusion and future work

This work investigated whether observable characteristics of a skill and of its repository are associated with cross-owner reuse in GitSkills, and how much of this predictability is an artifact of the dataset's construction. Three conclusions hold.

**First**, GitSkills has construction biases that affect predictive studies. The rule that selects the representative makes the probability of it being in `.claude/skills/`

grow from 14% to 92% with the number of copies, while the actual share of copies in that folder stays at 5–13%, so location and commit history are spurious predictors of reuse. In addition, about a quarter of the reused skills exist under a single owner and a few accounts concentrate the sample, so counting repositories overstates diffusion and performance depends on a few owners (AUC from 0.615 to 0.747 with the treatment).

**Second**, after excluding the identified construction-dependent shortcuts, the attributes predict cross-owner reuse with a modest signal confirmed on new data: on the owner-disjoint holdout, the Random Forest reaches an AUC of 0.731 (95% CI 0.716–0.745), and the HistGradientBoosting reduces the RMSLE of `owner_count` by 6% relative to the mean (0.625 against 0.664). Skill and repository contribute complementarily, and the signal is associated with stars and forks, with older last pushes and, in the regression, with the absence of a license. At a decision threshold the balanced accuracy is 0.66 and the precision for single-owner content is 0.28: the model ranks rather than classifies. These figures describe an owner-balanced sample, not the whole ecosystem, and do not extend to heavily concentrated owners. The exclusion does not remove every effect of the representative rule: it still determines which repository supplies stars, forks, license, last push and path, and the number of copies may influence that selection, so residual selection effects remain possible.

**Third**, under the evaluated model families and tuning budget, increasing model flexibility did not consistently improve predictive performance. This suggests that richer representations and improved measurements may be promising directions, although additional modeling gains cannot be ruled out. The claims are associative and cross-sectional: attributes and target share one snapshot, so the results do not evaluate prospective prediction of future reuse, and they identify neither what makes a skill more or less copied nor its quality.

**Future work.** (i) Compare observation caps with inverse-owner-frequency weighting, which keeps all contents and makes the target population explicit. (ii) Test robustness to the selection rule by choosing the representative independently of location preferences, which requires attributes of alternative copies. (iii) Evaluate prospective prediction with attributes dated before the copies appear (for example, repository state at an earlier snapshot). (iv) Model at the copy level, with information prior to reuse, to use attributes of all copies without leakage. (v) Group near-identical copies to mitigate the fragmentation of the target by exact hash. (vi) Replicate with later snapshots and study the temporal dynamics of diffusion. (vii) Investigate why repositories without a license host skills with more owners. (viii) Evaluate text representations within fixed location classes. (ix) Propose to the GitSkills maintainers indicators that separate the representative choice from repository attributes.

## References


[Agent Skills ] Agent Skills. Specification – agent skills. `https://agentskills.io/specification`. Accessed: 2026-10-27.

[Anthropic 2026] Anthropic (2026). Claude code. `https://claude.com/claude-code`. Agentic coding assistant used in the revision of the study.

[Breiman 2001] Breiman, L. (2001). Statistical modeling: The two cultures (with comments and a rejoinder by the author). *Statistical Science*, 16(3):199–231.

[Destefanis et al. 2027] Destefanis, G., Graziotin, D., Vaccargiu, M., and Ortu, M. (2027). GitSkills: A dataset of agent skills on GitHub. In *Proceedings of the 24th IEEE/ACM International Conference on Mining Software Repositories (MSR)*.

[Geirhos et al. 2020] Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673.

[Haefliger et al. 2008] Haefliger, S., Von Krogh, G., and Spaeth, S. (2008). Code reuse in open source software. *Management Science*, 54(1):180–193.

[Han et al. 2026] Han, T., Zhang, Y., Song, W., Fang, C., Chen, Z., Sun, Y., and Hu, L. (2026). SWE-Skills-Bench: Do agent skills actually help in real-world software engineering? *arXiv preprint arXiv:2603.15401*.

[Heinemann et al. 2011] Heinemann, L., Deissenboeck, F., Dreier, M., Göde, N., Hummel, B., and Jürgens, E. (2011). On the extents, origins, and problems of software reuse in open source software. In *Proceedings of the 14th International Conference on Fundamental Approaches to Software Engineering (FASE)*, pages 258–272. Springer.

[Kaufman et al. 2012] Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 6(4):1–21.

[Lapuschkin et al. 2019] Lapuschkin, S., Wäldchen, S., Binder, A., Montavon, G., Samek, W., and Müller, K.-R. (2019). Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications*, 10(1):1096.

[Lopes et al. 2017] Lopes, C. V., Maj, P., Martins, P., Sajnani, V., Di Rocco, J., Summer, H., and Baldi, P. (2017). Déjàvu: a map of code duplicates on GitHub. *Proceedings of the ACM on Programming Languages*, 1:1–28.

[Mockus et al. 2002] Mockus, A., Fielding, R. T., and Herbsleb, J. D. (2002). Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 11(3):309–346.

[Molnar 2020] Molnar, C. (2020). *Interpretable machine learning: A guide for making black box models explainable*. Independently published.

[Qin et al. 2023] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al. (2023). Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

[Roberts et al. 2017] Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929.

[Saeb et al. 2017] Saeb, S., Lonini, L., Jayaraman, A., Mohr, D. C., and Kording, K. P. (2017). The need to separate related samples in cross-validation: Error estimation for models that predict cross-subject traits. *GigaScience*, 6(4):gix019.

[Schick et al. 2023] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.

[Shen et al. 2023] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. (2023). Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:37819–37834.

[Shmueli 2010] Shmueli, G. (2010). To explain or to predict? *Statistical Science*, 25(3):289–310.

[Tafreshipour et al. 2025] Tafreshipour, M., Imani, A., Huang, E., de Almeida, E. S., Zimmermann, T., and Ahmed, I. (2025). Prompting in the wild: An empirical study of prompt evolution in software repositories. In *Proceedings of the 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR)*.

[Yao et al. 2023] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

[Zhang et al. 2026] Zhang, C., Liu, Y., Chen, X., and Ji, P. (2026). What keeps agent skills from being reusable? evidence from 138k SKILL.md files. *arXiv preprint arXiv:2608.08453*.

## A Reproducibility and execution

**Data, environment, seeds.** Public source: GitSkills on Hugging Face, fixed revision `ebab17454a7c236f8f26b183567f1a126f42e3f8` (July 2026; SHA-256 verified). Python 3.12, `pyproject.toml` and `uv.lock`. Seeds: sampling `20261009`; validation 2027–2029; holdout `20261110`; bootstrap 2027. The 200,000-content sample is deterministic (`md5(content_hash || ':20261009')`), not a simple random sample. Code and data tables: github.com/podorize/skillreuse.

**Quick reproduction (about 5 minutes, no 6.5 GB download).** `make quick` uses the small derived tables in `data/derived/` (9.5 MB), runs the main protocol with one repetition and re-runs the frozen models on the holdout into a separate directory; the holdout numbers reproduce exactly. A Colab notebook is in `notebooks/reproduce.ipynb`.

**Full pipeline.**

```
uv sync
make data owner feature-eda        # download, sample, owner targets, au
make controls primary robustness   # target-`repository_count` control,
make importance interpretation     # permutation importance, partial dep
make final holdout                 # nested tuning and freezing (4.6); h
make holdout-eval                  # single evaluation (4.7)
```

The frozen models are in `results/owner/final/final_config_*.json`, committed before the evaluation (git tag `frozen-before-holdout`).

`holdout-eval` writes `results/owner/holdout/evaluation_lock.json` and refuses to run again without `--allow-reevaluation`; any re-evaluation must be declared. Every table and figure maps to a CSV or JSON under `results/owner/` (and `results/controls/` for the `repository_count` control), each directory with a `manifest.json`; the targets of `make` name the step that produces each one. `make test` runs the 40 tests (excluded attributes, path patterns, paired tests and Holm, owner bootstrap, holdout disjointness, threshold selection).

**Deviations and limits.** Tuning used $2\times3$ folds and 6 candidates (planned $3\times3$ and 12) for computational cost, on a machine with 8 GB of memory. Numerical results may vary in the last digits across library versions. Earlier exploratory results (target `repository_count`, original external test) remain in the repository (`results/count_regression`, `results/discipline_benchmark`) only as history and support no conclusion of this report. The protocol document `paper/protocol.md` is in Portuguese.

## B Use of artificial intelligence tools

Claude Code [Anthropic 2026] (Anthropic) was used as an implementation and analysis assistant throughout the project: dataset audit, target and feature construction, validation and tuning code, holdout construction, tests, figures and drafting. The author defined the research questions, approved each methodological decision (owner-based target, excluded attributes, per-owner cap, protocol, holdout and single evaluation), reviewed the outputs and is responsible for the code, the claims and the interpretation. Automated tests check the numbers reported in the text against the result files.